Simulation of Timestamp Ordering (TO) Concurrency Control Algorithm

Aim

To simulate the Timestamp Ordering (TO) Concurrency Control Algorithm in a Freelance Marketplace using Python.

Domain

A Freelance Marketplace where multiple clients and freelancers access and update project information simultaneously. Timestamp Ordering (TO) is a concurrency control protocol that assigns a unique timestamp to every transaction. It ensures that all transactions execute according to their timestamps, preventing conflicts and maintaining database consistency.

Task 1: Schema Design

Create a table that represents the data item and stores the timestamps required for the Timestamp Ordering protocol.

Schema Design

Attribute	Description
Project_ID	Unique identifier of the project
Project_Name	Name of the freelance project
Budget	Budget allocated to the project
Read_TS	Highest timestamp of any transaction that has successfully read the data item
Write_TS	Highest timestamp of any transaction that has successfully written the data item

Initial Data

Project_ID	Project_Name	Budget	Read_TS	Write_TS
301	Website Developmen t	50000	0	0

Task 2: Insert Initial Values and Transactions

Initialize the data item and create two transactions with logical timestamps.

Data Item

Project_ID	Project_Name	Budget
		Timestamp T1 10
301	Website Development	T2 20
		50000

Transactions

Transaction

Transaction T1 is assigned timestamp 10.

Transaction T2 is assigned timestamp 20.

Since 10 < 20, Transaction T1 is older than Transaction T2.

Task 3: Simulate the Read Operation

Python Code

```
data_item = {
 "Project_ID": 301,
 "Project_Name": "Website Development",
 "Budget": 50000,
```

```
 "Read_TS": 0,
 "Write_TS": 0
}
transactions = {
 "T1": 10,
 "T2": 20
}
def read(transaction):
 ts = transactions[transaction]

 if ts < data_item["Write_TS"]:
 print(transaction, "Read Rejected")
 else:
 data_item["Read_TS"] = max(data_item["Read_TS"], ts)
print(transaction, "Read Successful")
 print("Current Budget =", data_item["Budget"])
read("T1")
```

```
data_item = {
    "Project_ID": 301,
    "Project_Name": "Website Development",
    "Budget": 50000,
    "Read_TS": 0,
    "Write_TS": 0
}

transactions = {
    "T1": 10,
    "T2": 20
}

def read(transaction):
    ts = transactions[transaction]

    if ts < data_item["Write_TS"]:
        print(transaction, "Read Rejected")
    else:
        data_item["Read_TS"] = max(data_item["Read_TS"], ts)
        print(transaction, "Read Successful")
        print("Current Budget =", data_item["Budget"])

read("T1")
```

```
T1 Read Successful
Current Budget = 50000
```

Task 4: Simulate the Write Operation

```
def write(transaction, new_budget):

 ts = transactions[transaction]

 if ts < data_item["Read_TS"] or ts < data_item["Write_TS"]:
print(transaction, "Write Rejected")
 else:
 data_item["Budget"] = new_budget
 data_item["Write_TS"] = ts
 print(transaction, "Write Successful")
 print("Updated Budget =", data_item["Budget"])

write("T2", 70000)
```

```
def write(transaction, new_budget):

    ts = transactions[transaction]

    if ts < data_item["Read_TS"] or ts < data_item["Write_TS"]:
        print(transaction, "Write Rejected")
    else:
        data_item["Budget"] = new_budget
        data_item["Write_TS"] = ts
        print(transaction, "Write Successful")
        print("Updated Budget =", data_item["Budget"])

write("T2", 70000)
```

```
T2 Write Successful
Updated Budget = 70000
```

Demonstration of Write Rejection

```
write("T1", 60000)
```

```
write("T1", 60000)
```

```
T1 Write Rejected
```

Task 5: Explanation of Timestamp Ordering Operations

1. The data item is initialized with a project budget of ₹50,000, Read_TS = 0, and

Write_TS = 0.

2. Transaction T1 performs a read operation. Since its timestamp (10) is greater than the current Write_TS (0), the read is successful, and Read_TS is updated to 10.
3. Transaction T2 performs a write operation. Since its timestamp (20) is greater than both Read_TS (10) and Write_TS (0), the write is successful. The project budget is updated from ₹50,000 to ₹70,000, and Write_TSbecomes 20.
4. Transaction T1 again attempts to write. Since its timestamp (10) is less than the current Write_TS (20), the write operation is rejected. This prevents an older transaction from overwriting newer data and maintains database consistency.